

Full Marks:50

SET:A

Time: 1Hour 10Minutes

Q1. What is the output?

```
try:
    raise ValueError
except ValueError:
    print("A")
    raise
```

- A)No Output
- B)B
- C)A, followed by propagation of the same exception
- D)A, and execution ends normally

Q2. What is the output?

```
def fun(x):
    if x == 0:
        return
    return x + fun(x - 1)
```

```
print(fun(3))
```

- A)6
- B)3
- C)None
- D)5

Q3. What happens here?

```
try:
    raise ValueError("Wrong value")
except ValueError as e:
    print(e)
```

- A)ValueError
- B)Wrong Value
- C)Nothing
- D)e

Q4. What will be the output?

```
x = 10
```

```
def outer():
    x = 20
```

```
    def inner():
```

```
    global x
    x = 30
```

```
    inner()
    print(x)
```

```
outer()
print(x)
```

- A)30
- 30
- B)20
- 30
- C)20
- 10
- D)30
- 20

Q5. What is the output?

```
def outer():
    x = 10
```

```
    def inner():
        return x + 5
```

```
    return inner
```

```
f = outer()
print(f())
```

- A)NameError
- B)10
- C)None
- D)15

Q6. What will be the output?

```
def fun(x):
    try:
        if x < 0:
            raise ValueError("negative")
        return x
    except ValueError:
        return 100
    finally:
```



```
print("cleanup")
```

```
print(fun(-5))
```

A)cleanup

-5

B)negative

cleanup

C)cleanup

100

D)100

cleanup

Q7. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a
```

```
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

A)ImportError

B)10

10

C)Module loaded

10

D)Module loaded

Module loaded

10

Q8. What is the major difference between the following?

```
import math
```

and

```
from math import sqrt
```

A)There is no difference

B)Both import only sqrt

C)First imports only sqrt

D)First provides the module namespace; second imports sqrt directly

Q9. What will happen?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        nonlocal y
```

```
        y = 20
```

```
    inner()
```

```
outer()
```

A)NameError

B)20

C)SyntaxError

D)10

Q10. What happens here?

```
def fun(a):
```

```
    return a
```

```
print(fun())
```

A)TypeError

B)0

C)None

D)NameError

Q11. What will be printed?

```
def fun():
```

```
    print("A")
```

```
    return
```

```
    print("B")
```

```
fun()
```

A)Nothing

B)A

C)A B

D)B

Q12. What will be the output?

```
import math
```

```
print(math.sqrt(25))
```

A)5

B)5.0

C)sqrt(25)

D)25



Q13. What happens here?

```
class MyError(Exception):  
    pass
```

```
def test():  
    raise MyError("Something went  
wrong")
```

```
try:  
    test()  
except MyError as e:  
    print(type(e).__name__)
```

- A)MyError
- B)Exception
- C)TypeError
- D)Something went wrong

Q14. What is the output?

```
def fun():  
    try:  
        return 10  
    finally:  
        return 20
```

```
print(fun())  
A)RuntimeError  
B)10  
C)None  
D)20
```

Q15. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))  
A)15  
B>Error  
C)10  
D)5
```

Q16. What is the output?

```
numbers = [1, 2, 3, 4, 5]  
  
result = list(  
    map(  
        lambda x: x * 2,  
        filter(lambda x: x % 2 == 0,  
numbers)
```

```
)  
)  
  
print(result)
```

- A)[1, 3, 5]
- B)[4, 8]
- C)[2, 4]
- D)[2, 4, 6, 8, 10]

Q17. What will be printed?

```
try:  
    print(10 / 0)  
except ZeroDivisionError:  
    print("A")  
except Exception:  
    print("B")
```

- A)A B
- B)B
- C)Nothing
- D)A

Q18. What happens when this code runs?

```
def first():  
    second()
```

```
def second():  
    third()
```

```
def third():  
    raise RuntimeError("Failure")
```

```
try:  
    first()  
except RuntimeError:  
    print("Handled")  
A)Handled  
B)RuntimeError cannot propagate through  
functions  
C)TypeError  
D)NameError
```

Q19. What will be printed?

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```



```
print(f())
```

- A)10
- B)None
- C)NameError
- D)20

Q20. Which object keeps track of modules that have already been imported during the current Python process?

- A)__module__
- B)sys.imports
- C)sys.path
- D)sys.modules

Q21. Why is

try:

```
    print(10 / 0)
```

except Exception:

```
    print("A")
```

except ZeroDivisionError:

```
    print("B")
```

codes's second handler never reached?

- A)Exception is a superclass of ZeroDivisionError
- B)ZeroDivisionError occurs before Exception
- C)except blocks execute randomly
- D)ZeroDivisionError is not an exception

Q22. What is the output?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        nonlocal x
```

```
        x = x + 5
```

```
        return x
```

```
    inner()
```

```
    return x
```

```
print(outer())
```

- A)10
- B)5
- C)15
- D)UnboundLocalError

Q23. What will happen?

```
def fun(n):
```

```
    if n == 0:
```

```
        return 1
```

```
    return n * fun(n)
```

```
print(fun(3))
```

- A)6
- B)TypeError
- C)RecursionError
- D)3

Q24. Suppose sample.py contains:

```
x = 10
```

```
def display():
```

```
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)
```

```
sample.display()
```

- A)10
- Hello
- B)x
- display
- C>Error
- D)Only Hello

Q25. What will be the output?

```
x = 10
```

```
def fun():
```

```
    print(x)
```

```
    x = 20
```

```
fun()
```

- A)UnboundLocalError
- B)20
- C)10
- D)NameError

Q26. What will be printed?

```
def make_counter():
```

```
    count = 0
```



```
def counter():
    nonlocal count
    count += 1
    return count
```

```
return counter
```

```
a = make_counter()
b = make_counter()
```

```
print(a())
print(a())
print(b())
print(a())
```

A)1

1

1

1

B)1

2

3

4

C)0

1

0

2

D)1

2

1

3

Q27. What will be printed?

```
def fun(x=[]):
    x.append(10)
    return x
```

```
a = fun()
b = fun()
```

```
print(a)
print(b)
```

A)[10, 10]

[10]

B>Error

C)[10]

[10]

D)[10]

[10, 10]

Q28. What happens here?

```
from math import sqrt
```

```
print(sqrt(36))
```

A)6.0

B)NameError

C)36

D)6

Q29. What will be printed?

```
class InvalidAgeError(Exception):
    pass
```

```
def check_age(age):
```

```
    if age < 18:
```

```
        raise InvalidAgeError("Age is
too low")
```

```
    return "Allowed"
```

```
try:
```

```
    print(check_age(15))
```

```
except InvalidAgeError as e:
```

```
    print(type(e).__name__)
```

```
    print(e)
```

A)Nothing, because custom exceptions cannot be caught

B)Exception

Age is too low

C)InvalidAgeError

Age is too low

D)InvalidAgeError

15

Q30. if test.py has

```
print("A")
```

```
if __name__ == "__main__":
```

```
    print("B")
```

If test.py is executed directly, what will be printed?

A)A followed by B

B)A

C)Nothing

D)B



Q31. What is the output?

```
def fun():  
    try:  
        print("A")  
        raise ValueError  
    except ValueError:  
        print("B")  
        return 10  
    else:  
        print("C")  
        return 20  
    finally:  
        print("D")
```

print(fun())

A)A

B

D

10

B)A

B

C

D

10

C)A

C

D

20

D)A

B

10

D

Q32. What happens when this code executes?

```
def fun():  
    print(variable)  
    variable = 50
```

variable = 10

fun()

A)NameError

B)UnboundLocalError

C)50

D)10

Q33. What is the purpose of this statement?

```
if __name__ == "__main__":
```

A)It prevents all imports

B)It identifies code intended to run when the file is executed directly

C)It prevents syntax errors

D)It creates the main function

Q34. What is the output?

```
def fun(x):  
    return x * 2
```

a = fun

print(a(5))

A>Error

B)5

C)10

D)func(5)

Q35. What will normally happen?

import math

import math

A)ImportError

B)The module is normally loaded and executed twice

C)The module is normally loaded once and reused

D)SyntaxError

Q36. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(  
    lambda a, b: a * b,  
    numbers  
)
```

print(result)

A)9

B)24

C)0

D)10

Q37. What is the output?



```
def fun(x):
```

```
    x += 10
```

```
    return x
```

```
x = 5
```

```
y = fun(x)
```

```
print(x, y)
```

A)15 5

B)15 15

C)5 5

D)5 15

Q38. Suppose the current directory contains a file named random.py.

What potential problem can occur with:

```
import random
```

A)Python will always reject it

B)The local file can shadow the intended standard-library module

C)random cannot be imported

D)Python automatically renames the local file

Q39. What is the output?

```
try:
```

```
    print(10 / 0)
```

```
except Exception:
```

```
    print("A")
```

```
except ZeroDivisionError:
```

```
    print("B")
```

A)SyntaxError

B)A B

C)B

D)A

Q40. What will be printed?

```
try:
```

```
    x = int("10")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

A)B C

B)A

C)B

D)A C

Q41. What happens?

```
def fun(a, b):
```

```
    return a + b
```

```
print(fun(10, a=20))
```

A)30

B)20

C)TypeError

D)SyntaxError

Q42. What will happen?

```
def fun(a, b=10):
```

```
    return a + b
```

```
print(fun(b=5, a=20))
```

A)25

B)15

C)TypeError

D)SyntaxError

Q43. What will be printed?

```
def fun(a, b):
```

```
    return a + b
```

```
print(fun(10, b=20))
```

A)SyntaxError

B)20

C)TypeError

D)30

Q44. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":
```

```
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

A)Nothing

B)A and B

C)B

D)A



Q45. What does the following function return?

```
def fun():  
    print("Hello")
```

```
x = fun()
```

```
print(x)
```

- A)Hello
- B)Hello None
- C)Error
- D)None

Q46. What will be printed?

```
x = 10
```

```
def outer():
```

```
    x = 20
```

```
    def inner():
```

```
        print(x)
```

```
    x = 30
```

```
    inner()
```

```
outer()
```

- A)10
- B)20
- C)30
- D)NameError

Q47. What is the output?

```
try:
```

```
    x = int("abc")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

```
A)B
```

```
B)A C
```

```
C)B C
```

```
D)A
```

Q48. What is the output?

```
def fun(a=10, b=20):
```

```
    return a - b
```

```
print(fun(30))
```

```
A)-10
```

```
B)30
```

```
C)20
```

```
D)10
```

Q49. What will be printed?

```
def fun():
```

```
    try:
```

```
        return 10
```

```
    finally:
```

```
        print("Finally")
```

```
print(fun())
```

```
A)10 followed by Finally
```

```
B)Finally only
```

```
C)10 only
```

```
D)Finally followed by 10
```

Q50. What happens when this function is called?

```
def fun(a=10, b):
```

```
    return a + b
```

```
A)TypeError
```

```
B)NameError
```

```
C)SyntaxError
```

```
D)It executes correctly
```

